

Programming Assignment 3

Retrieval, Attention, and LLM

Deadline: 02/04/2026 11:59PM

Overview

In this assignment, we will study how language models select the correct tool given a natural language query. This problem appears in many real-world systems such as LLM agents choosing which API to call, assistants deciding between tools (search, calculator, weather, etc.), retrieval systems selecting relevant documents before response generation.

Formally, for each query, you are given a fixed set of ~100 tools, each tool comes with a description. Your goal is to retrieve the correct tool corresponding to the query. This setup can be seen as a simplified version of tool selection in LLM-based systems, where the model must decide which external functionality to invoke. Understanding how retrieval works in this setting is crucial for building reliable and efficient LLM applications.

Structure of assignment

The assignment is divided into three parts:

- **Part 1:** Classical retrieval methods
- **Part 2:** Attention-based retrieval and positional effects.
- **Part 3:** Identifying retrieval-relevant attention heads

Each part builds on the previous one, moving from standard retrieval pipelines to analyzing the internal behavior of LLMs.

Part 1: Classical Retrieval

In this part, you will implement standard retrieval methods where queries and tools are encoded independently, and similarity is used to retrieve relevant tools.

[Task] Given a query and a set of tools:

- Encode the query and tools independently
- Compute similarity between the query and each tool
- Retrieve the top-k most relevant tools

[Deliverable] Evaluate the following retrieval methods:

- BM25 (sparse retrieval baseline)
- msmarco-MiniLM (dense retrieval)
- UAE-large-v1 (dense retrieval)

Each method should return a ranked list of tools for every query. Report Recall@1 and Recall@5 for each method.

Part 2: Lost-in-the-middle

In this part, we move to a different setting where the query and all candidate tools are placed together in a single prompt and processed jointly by a language model.

The goal is to understand whether attention inside the model can be used as a retrieval signal, and how this signal behaves as the position of the relevant tool changes.

[TASK] For each query, you are given a prompt that contains: a list of tools and the query appended at the end. Our task is to use the model's attention to assign a score to each tool.

Concretely, for every query:

1. run the model on the full prompt
2. extract the attention matrices
3. compute how much attention flows from query tokens to each tool
4. use this to score and rank all tools

This produces a ranked list of tools for each query, similar to Part 1, but now the scoring comes from the model's internal attention.

[Code Template] You are provided with a template in `run2.py`.

The main components are:

PromptUtils: constructs the full prompt containing all tools and the query

doc_spans: gives the token span corresponding to each tool in the prompt

attentions: returned by the model, containing attention weights for all layers and heads

You will need to complete:

- `get_query_span(...)`: Identify the token span corresponding to the query in the prompt
- `query_to_docs_attention(...)`: Compute a score for each tool using attention from query tokens to tool tokens
- ranking and evaluation
- Use the scores to rank tools and compute metrics
- `analyze_gold_attention(...)`: Aggregate results across queries and generate a plot

Note: follow `# TODO` mentioned in the template and complete them accordingly.

[Deliverable] Using the attention-based scores, rank tools for each query in `test_queries` dataset. Report: `Recall@1` and `Recall@5`

[Deliverable] In addition to retrieval performance, we will analyze how attention to the correct(gold) tool varies with its position in the prompt.

In the code, for each query, we record: 1) position of the correct tool, 2) attention score assigned to it and 3) its rank

Using this data across all queries, generate a plot where: x-axis represents the position of the correct tool and y-axis represents the attention it receives.

The plot should be aggregated across queries to reveal any consistent trends.

We will need to complete:

- `analyze_gold_attention(...)`
 - The plot should be stored under name “plot2/gold_attention_plot.png”

Part 3: Retrieval Heads

In Part 2, you used attention aggregated across all heads to score tools. In this part, you will investigate whether a small subset of attention heads can act as effective retrievers.

The key question is: Are there specific heads in the model that are particularly good at identifying the relevant tool?

[Task] This part has two stages: selecting useful heads using training data, and then using only those heads at test time.

Phase 1: Head Selection

Using the training queries, identify a subset of attention heads that are useful for retrieval.

During training, you will modify the function: `select_retrieval_heads(...)` in `code3.py`.

This function should:

- construct prompts in the same way as Part 2 [done in template]
- run the model and extract attention [done in template]
- analyze how each head distributes attention from the query to tools
- aggregate this information across multiple queries
- return a list of **K heads** (e.g., $K = 20$), each represented as `(layer_id, head_id)`

One reasonable approach is to identify heads that **consistently assign high attention to the correct tool**, for example heads that tend to rank the gold tool at the top across many queries. You are free to design your own scoring strategy.

Phase 2: Retrieval Using Selected Heads

Using the selected heads:

- compute attention-based scores for tools (similar to Part 2) but **only use the selected heads** when computing scores
- rank tools for each query

[Code] Provided with templates in `run3.py` and `code3.py`.

- `select_retrieval_heads(...)`: Uses training queries to identify important heads - done in Phase 1
- `query_to_docs_attention_heads(...)`: Computes tool scores using only selected heads

- `get_query_span(...)`: Identifies the query tokens in the prompt

[Deliverables]

- List of selected heads: [(layer_id, head_id), ...]
- Recall@1 and Recall@5 for test queries.
- A short analysis describing:
 - how you selected the heads in Phase 1.
 - Mention the selected heads.
 - how performance of recall in test queries compares with Part 2 and Part 1
 - what this suggests about the role of different attention heads

[BONUS] Test with `max_heads` as 10, 20, 30. How does this affect the recall performance in test queries

[BONUS] Try different approaches to perform head selection in Part 3.1. Mention them briefly, along with their recall performance.

Dataset and code

https://github.com/deekshakoul/CS728_PA3

Do not change the model and seed present in code.

Mention in report

Complete the TODOs in code and in report mention the following (these are the same pointers mentioned under Deliverables in each part):

Part 1. A table with recall@1 and recall@5 for three baselines

Part 2.1 A table with recall@1 and recall@5.

Part 2.1 Plot to visualize position effects on attention based ranking.

Part 3.1 Brief description about head selection strategy along with the heads that were finally chosen.

Part 3.2 A table with recall@1 and recall@5.

Part 3.3 Compare the recall performance for Part 1., 2. and 3.

Submission Protocol:

Moodle will be used for the submission of the assignment.

- All the findings need to be put down into a report file.
- Complete the “# TODO” and don’t change the function names.
- **Do not add any jupyter notebook.**
- **Complete the assignment in a group of 4.**
- Zip all the code and the asset files, if any, along with the report into a single file and then submit it on Moodle. Name the compressed file as [Roll1_roll2.(zip/tar.gz)]
- If the code or assets does not fit in the Moodle submission limit, they can be offloaded onto Google Drive and working links provided in the report.

Note - Only one member of the team should submit the assignment on Moodle. **Teams can have up to four people.**